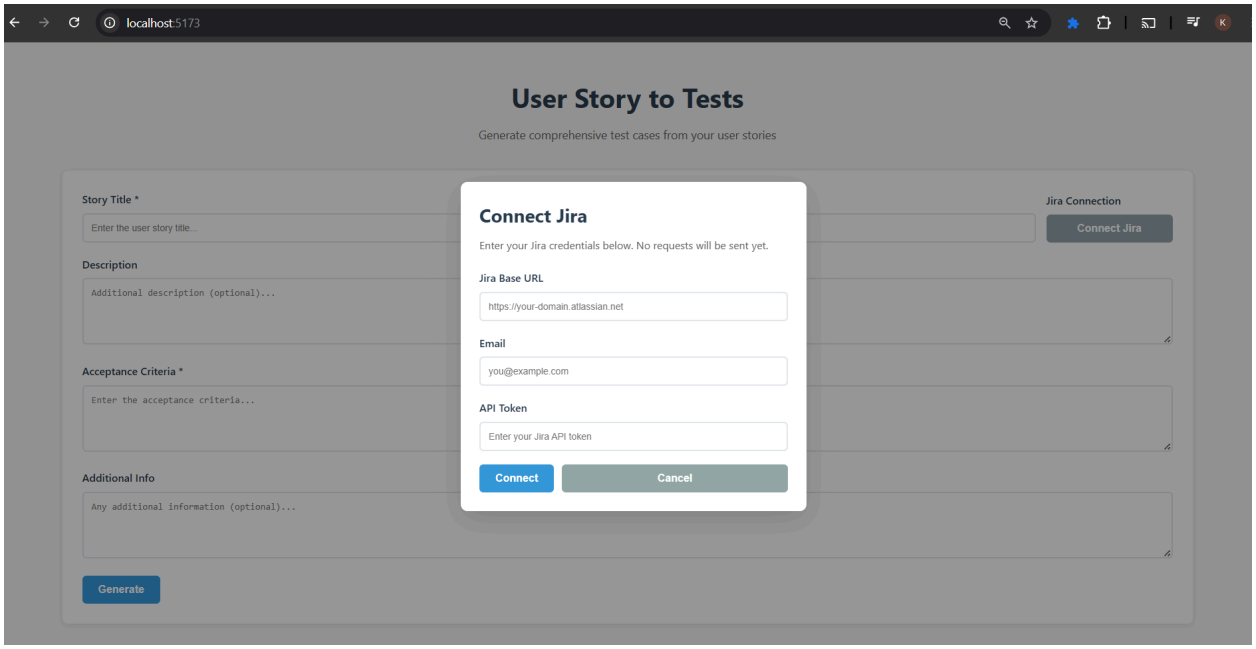
Testcase Generator

Step 1: Added “Connect Jira” button and a dialog box in the UI

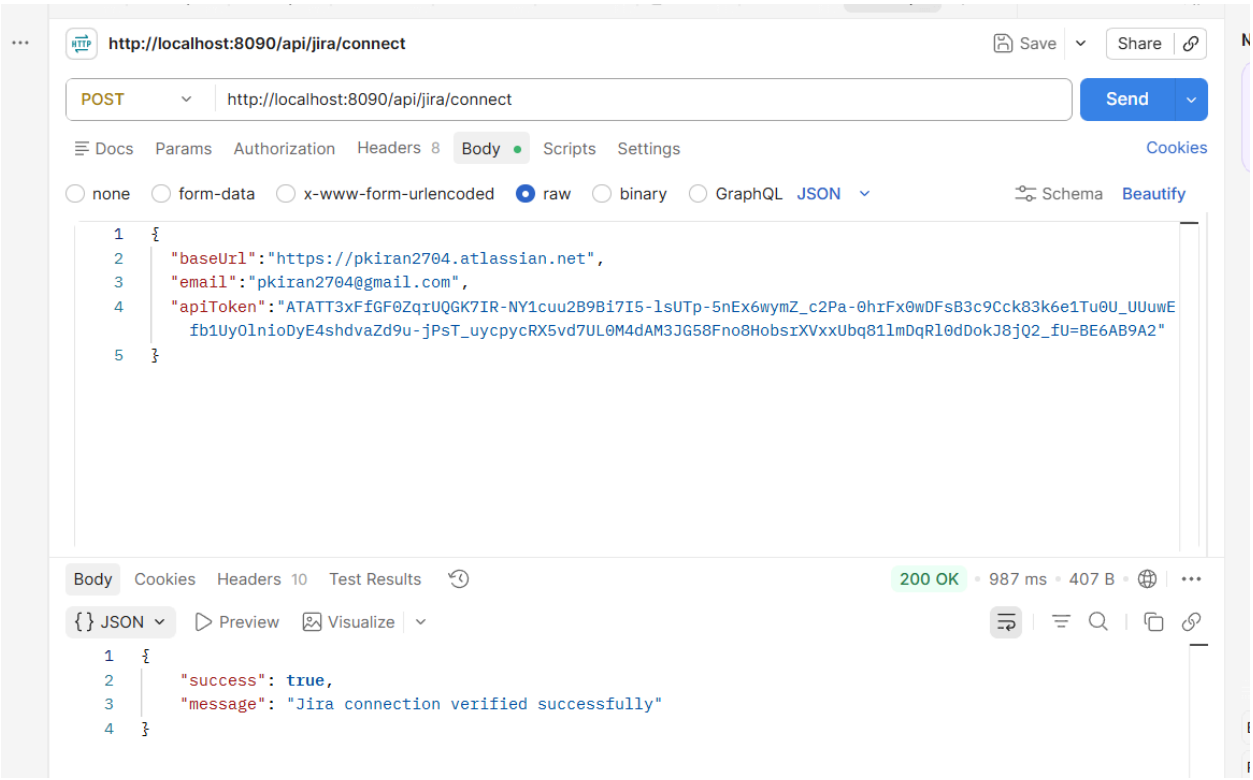


Step 2: Backend API

- Created a new router file backend/src/routes/jira.ts

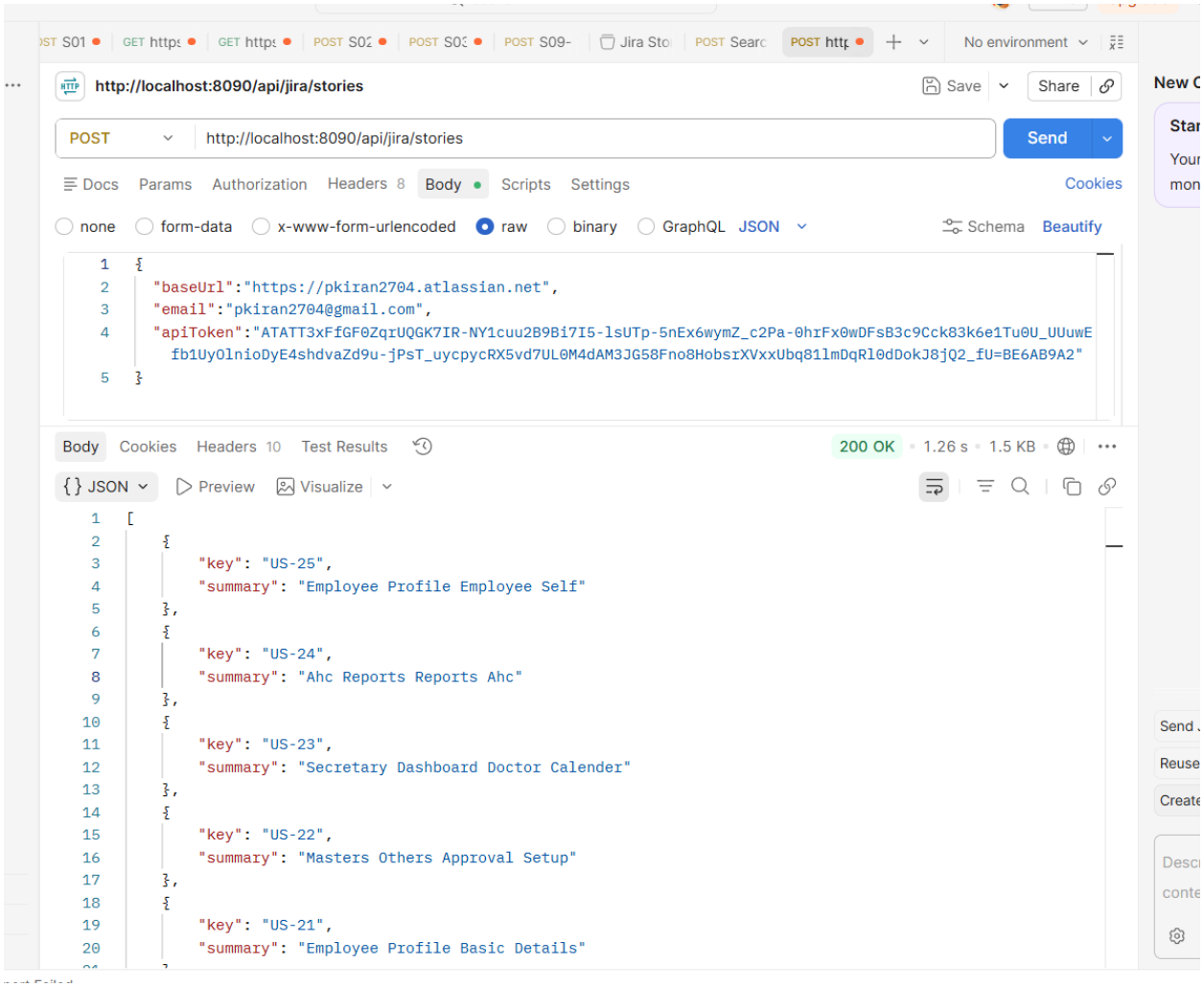
Step 3: Connect to Jira

- Added a new endpoint: POST http://localhost:8090/api/jira/connect
- It accepts: baseUrl, email, apiToken
- Authenticate against the Jira REST API
- Return: true/false, message



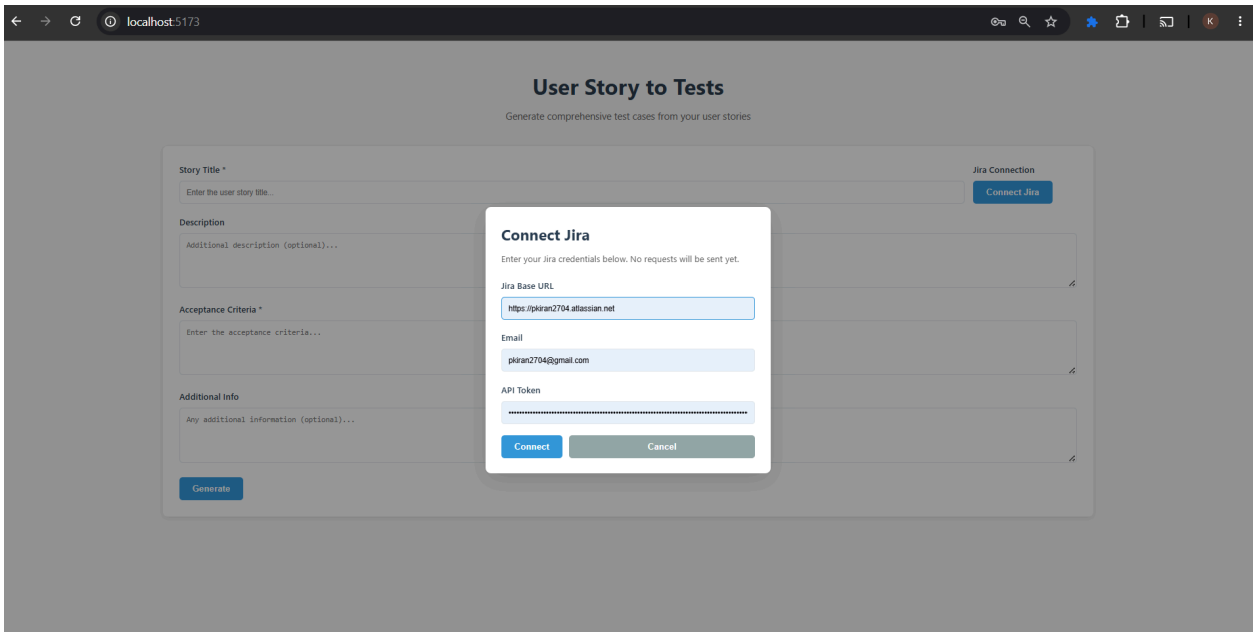
Step 4: Backend: Fetch User Stories

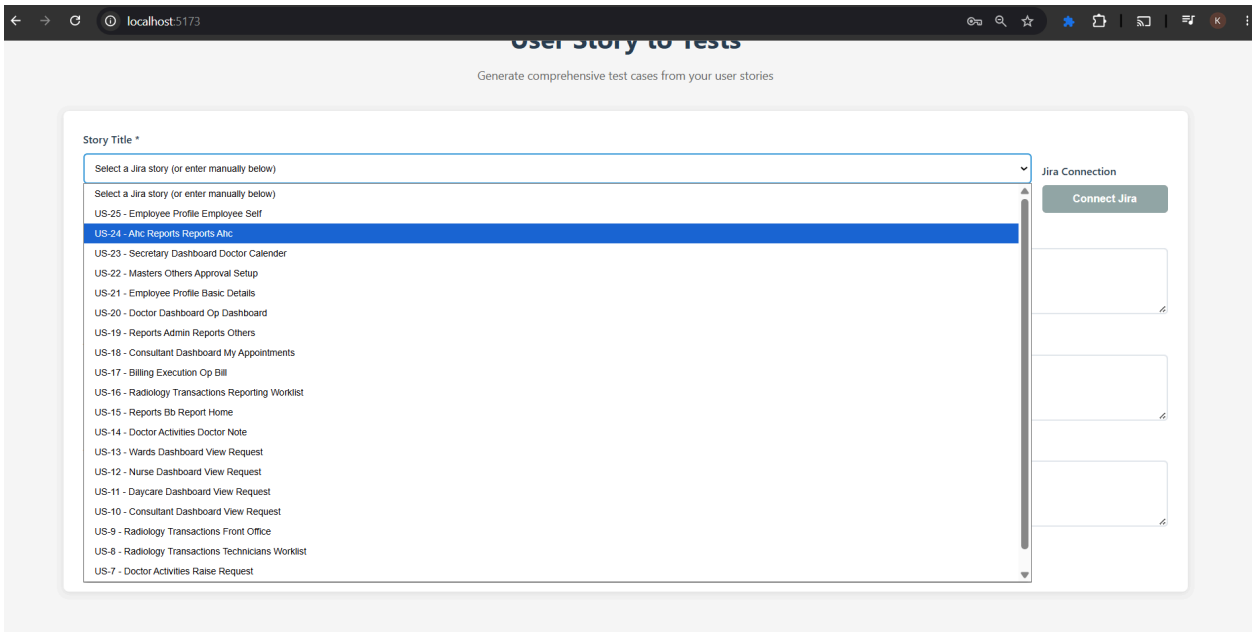
- Added a new endpoint: POST /api/jira/stories
- Input: baseUrl, email, apiToken
- Use the Jira Search REST API.
- Fetch the latest Story issues using JQL.
- Return a simplified JSON array containing only:key, summary
- Reuse the same authentication mechanism already implemented for /api/jira/connect.



Step 5: Frontend - Add API and Populate Dropdown

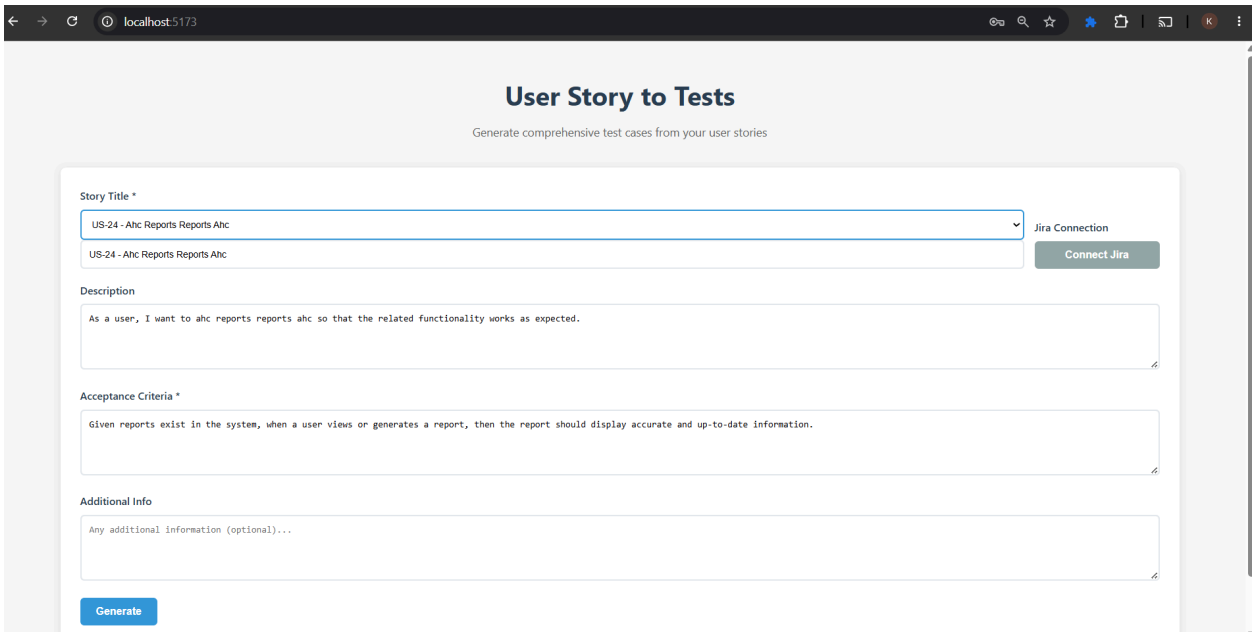
- Added a new function inside the existing frontend API layer that calls: POST /api/jira/stories
- Added Dropdown with the list of USer Stories

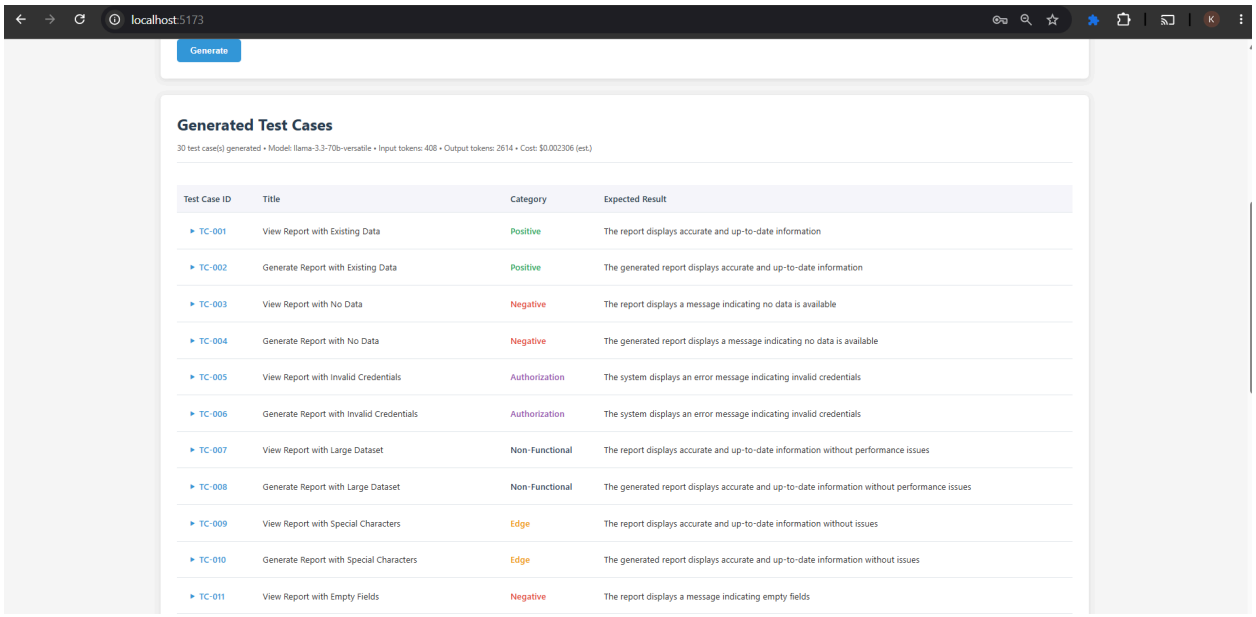




Step 6: Backend Story Details Endpoint & Frontend Wiring

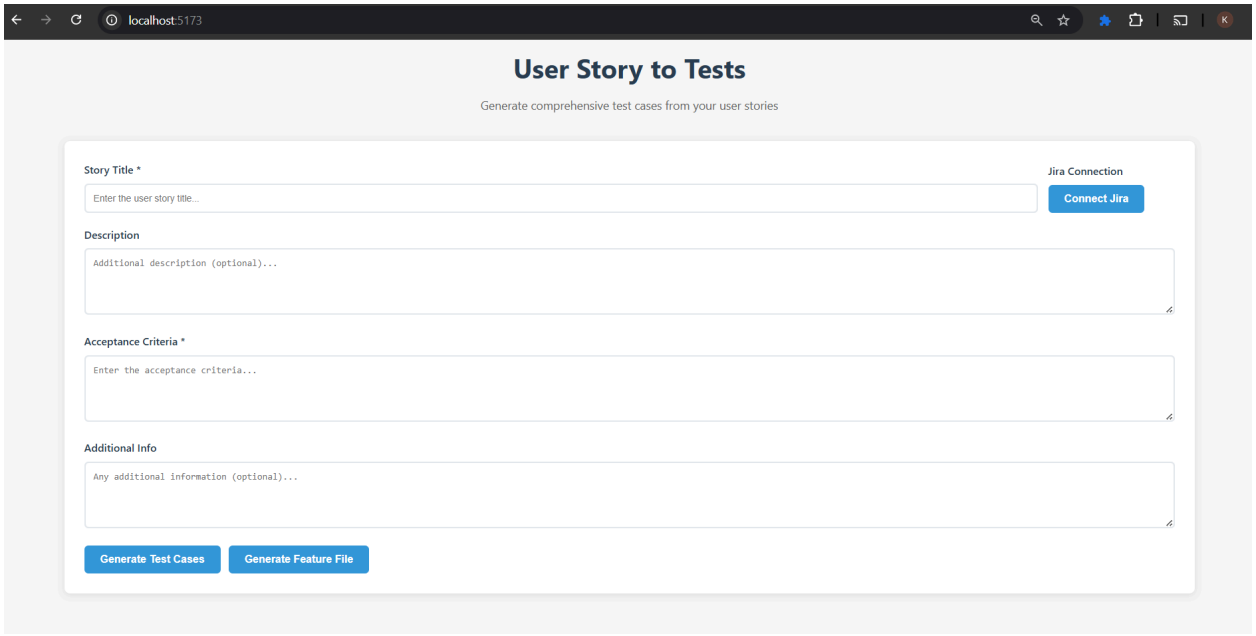
- Added a new endpoint: GET /api/jira/story/:key
- Call the Jira REST API to retrieve the full details for the selected issue.
- Return only the fields required by the current UI:key, summary, description, acceptanceCriteria
- UI calls the backend endpoint to retrieve the full story details.





Step 7: Added “Generate Feature File” button in UI

- Added a new button for Feature File Generation
- Modified Frontend APIs to have 2 values for the generationType parameter: testcase , feature
- Wired the buttons with the respective parameter values



Step 8: Update Backend

- Made changes to accept new generationType parameter
- Add support for a second prompt when generationType = "feature"
- The feature prompt instruct the LLM to generate a valid Gherkin Feature File
- And updated the generate.ts to switch the prompts based on user input